

Comparing Agent Trajectories: Edit Distance, by Hand

Why exact-match is the wrong tool for a *sequence* of tool calls — and how Levenshtein distance scores the path, one cell of a table at a time.

What this document does. It takes one tiny agent run from Module 3.0 — an agent whose trajectory is the tool sequence `[search, lookup, summarize]` measured against a reference (gold) path `[search, summarize]` — and computes, by hand, the *edit distance* between the two. We fill the entire dynamic-programming table cell by cell, read off the distance ($= 1$, a single deletion), turn it into a normalized similarity ($1 - \frac{1}{3} = 0.667$), and then contrast it with the order-agnostic precision/recall the module also mentions. Every number below is reproduced by the small Python program in Appendix A, so the table is exact and self-consistent.

Where this sits. This is **Topic 1 of Module 3.0** (*From RAG Eval to Agent Eval*) — the module that draws the line between *behavioral* eval (did the agent reach the goal?) and *trajectory* eval (did it take a sensible path?). Its companions: Module 3.1 (single-agent tool-F1 and trajectory scoring, where the set-based metric foreshadowed here is developed in full) and Module 3.2 (trajectory eval for multi-agent traces). The module’s own code lists `edit_distance: levenshtein(tools_used, tools_expected)` as a trajectory metric — this document is that one line, worked out by hand.

Concepts covered, in order: a trajectory as a sequence of tool calls · why exact string match fails when multiple correct paths exist · the three edit operations (insert, delete, substitute) · the Levenshtein DP recurrence · filling the full DP table · normalized trajectory similarity $1 - \frac{d}{\max(\text{len})}$ · set-based tool precision / recall / F1 (order-agnostic) · when order matters vs. when only the set matters · behavioral vs. trajectory axes (right answer, wasteful path).

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: two tool sequences, small enough to count by hand

In RAG eval the output is a single string and you score it with one number. In *agent* eval the output is a **trajectory** — an ordered sequence of tool calls the model chose, one after another. Module 3.0 is blunt about the consequence: the same task can be solved by *different correct paths*, so you cannot grade a trajectory by checking whether it equals the gold path *exactly*. Exact-match gives 0 or 1 and throws away every shade of “almost right.”

What we want instead is a *distance*: how many single-step edits turn the agent’s sequence into the reference sequence? That is the **Levenshtein (edit) distance**, and it is the metric the module lists in its trajectory-eval code.

The two sequences. We use deliberately tiny, hand-followable sequences. The agent did one extra step (a `lookup`) that the reference path skips.

Quantity	Symbol	Value here
Actual trajectory (tool sequence)	a	[search, lookup, summarize]
Reference / gold path	b	[search, summarize]
Length of actual	$n = a $	3
Length of reference	$m = b $	2
Cost of insert / delete	—	1 each
Cost of substitute (mismatch)	—	1
Cost of a match ($a_i = b_j$)	—	0

Notation. Tool calls are compared as whole tokens: two steps are “equal” iff they call the same tool (`search` = `search`). We write $d[i][j]$ for the edit distance between the first i tools of a and the first j tools of b . The answer we are after is $d[n][m] = d[3][2]$.

Intuition

Think of the agent’s trajectory as a word and the gold path as another word, where each *letter* is a whole tool call. Exact-match asks “are these the same word?” — a yes/no that collapses “off by one tool” and “completely wrong” into the same verdict. Edit distance asks the gentler, more useful question: “how many one-step corrections turn the agent’s word into the reference word?” An agent that did everything right plus one wasteful `lookup` is one delete away from gold — and the metric should say exactly that.

2 Step 1 — the three edits and the recurrence

To turn one sequence into another we are allowed three unit-cost moves, applied one tool at a time:

- **Delete** a tool from a (the agent took a step the reference does not) — cost 1.
- **Insert** a tool into a (the reference has a step the agent skipped) — cost 1.
- **Substitute** one tool for another (the agent called the wrong tool in that slot) — cost 1; or, if the two tools already match, cost 0.

The trick that makes this computable is the observation that an optimal alignment of $a[1..i]$ with $b[1..j]$ must end in exactly one of those three moves on the *last* pair of positions. That gives the dynamic-programming recurrence:

$$d[i][j] = \min \begin{cases} d[i-1][j] + 1 & (\text{delete } a_i) \\ d[i][j-1] + 1 & (\text{insert } b_j) \\ d[i-1][j-1] + \mathbf{1}[a_i \neq b_j] & (\text{substitute / match}) \end{cases} \quad (1)$$

with the base cases $d[i][0] = i$ (delete all i tools to reach the empty sequence) and $d[0][j] = j$ (insert all j tools from nothing). Here $\mathbf{1}[a_i \neq b_j]$ is 0 when the two tools are identical and 1 otherwise.

Mini-example — this operation in isolation

The substitution term is the only one that “looks back diagonally,” and it is where a *match* earns its keep. If $a_i = b_j$, the indicator is 0, so $d[i][j] = d[i-1][j-1]$: a matching tool is free and simply carries the previous diagonal cost forward. If they differ, the same diagonal step costs +1. Two single tools: `[search]` vs. `[search]` gives $d[1][1] = d[0][0] + 0 = 0$ (a free match); `[search]` vs. `[lookup]` gives $d[1][1] = d[0][0] + 1 = 1$ (one substitution).

3 Step 2 — fill the table cell by cell

We build an $(n+1) \times (m+1)$ grid. Rows are prefixes of the actual sequence a (top to bottom: ε , `search`, `lookup`, `summarize`); columns are prefixes of the reference b (left to right: ε , `search`, `summarize`). Fill the first row and column with the base cases, then sweep left-to-right, top-to-bottom, taking the minimum of the three incoming arrows at each cell.

$d[i][j]$	ε	<code>search</code>	<code>summarize</code>
ε	0	1	2
<code>search</code>	1	0	1
<code>lookup</code>	2	1	1
<code>summarize</code>	3	2	1

Walk the two interesting cells:

Cell (`search`, `search`), value 0. Tools match, so the diagonal term is $d[0][0] + 0 = 0$; the delete/insert terms are $d[0][1] + 1 = 2$ and $d[1][0] + 1 = 2$. The minimum is the free diagonal: $\min(2, 2, 0) = 0$. ✓

Cell (`summarize`, `summarize`), value 1 — the answer. Tools match, so the diagonal term is $d[\text{lookup}][\text{search}] + 0 = 1 + 0 = 1$; delete gives $d[\text{lookup}][\text{summarize}] + 1 = 1 + 1 = 2$; insert gives $d[\text{summarize}][\text{search}] + 1 = 2 + 1 = 3$. The minimum is $\min(2, 3, 1) = 1$. ✓

So the bottom-right cell is

$$\boxed{d[3][2] = 1} \tag{2}$$

One deletion, read off the table. The cheapest alignment matches `search`→`search` (free), *deletes* the agent’s `lookup` (cost 1), then matches `summarize`→`summarize` (free). Total 1 — exactly the one wasteful step the agent inserted. The table did not just give us a number; the path through it tells us *which* step was extra.

Intuition

Every cell answers a smaller version of the same question, and the recurrence reuses those answers instead of recomputing them — that is why a problem with exponentially many possible alignments collapses to an $n \times m$ grid you can fill by hand. The final distance is never larger than $\max(n, m)$ (you can always delete everything and re-insert), and it is 0 only when the two sequences are identical. Our 1 sits at the cheap end: “right path, one redundant call.”

4 Step 3 — from distance to a similarity score

A raw distance of 1 means little until you know how long the sequences were — one edit out of two tools is worse than one edit out of twenty. We normalize by the longer of the two lengths, which bounds the score to $[0, 1]$:

$$\text{sim}(a, b) = 1 - \frac{d[n][m]}{\max(n, m)}. \quad (3)$$

For our run, $\max(n, m) = \max(3, 2) = 3$, so

$$\text{sim} = 1 - \frac{1}{3} = \frac{2}{3} \approx \mathbf{0.667}.$$

A similarity of 0.667 reads as “two-thirds of the way from worst-case to a perfect match” — the agent’s path is mostly right, dinged once for the redundant `lookup`. Identical sequences score 1.0 (distance 0); a totally disjoint path of the same length scores 0.

Mini-example — this operation in isolation

Plug the two extremes into Equation (3). *Perfect agent*: $\text{actual} = \text{gold} \Rightarrow d = 0 \Rightarrow \text{sim} = 1 - 0/3 = 1.0$. *Wasteful but correct agent* (Module 3.0’s “Agent B,” a 9-step path against a 3-step gold): the edit distance works out to 6, so $\text{sim} = 1 - 6/9 = 0.333$. Same final answer as the 3-step agent, less than half the trajectory similarity — the metric sees the cost bomb the behavioral verdict misses.

5 Step 4 — when order matters vs. when only the set matters

Edit distance is **order-aware**: it cares that `search` came before `summarize`. Module 3.0 also lists an **order-agnostic** family — tool precision and recall over the *set* of tools used. The two answer different questions, and it pays to know which you are asking.

For our example, treat the sequences as sets. The agent used $\{\text{search}, \text{lookup}, \text{summarize}\}$; the gold set is $\{\text{search}, \text{summarize}\}$. Their intersection is $\{\text{search}, \text{summarize}\}$, of size 2:

$$\begin{aligned} \text{precision} &= \frac{|\text{used} \cap \text{gold}|}{|\text{used}|} = \frac{2}{3} \approx 0.667, & \text{recall} &= \frac{|\text{used} \cap \text{gold}|}{|\text{gold}|} = \frac{2}{2} = 1.0, \\ \text{tool-F1} &= \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \cdot \frac{2}{3} \cdot 1}{\frac{2}{3} + 1} = \frac{4/3}{5/3} = \frac{4}{5} = \mathbf{0.8}. \end{aligned}$$

Recall is a perfect 1.0 — the agent used *every* gold tool. Precision is 0.667 — it used one tool too many. This is the same “one redundant step” the edit distance caught, but scored on a different axis. **Module 3.1 builds tool-F1 out in full**; this is the seed.

Watch out

Set-based precision/recall is *blind to order and to repeats*. Consider an agent that calls `[refund, crm_lookup]` when the gold path is `[crm_lookup, refund]` — issuing the refund *before* looking up the customer. As sets these are identical, so $\text{precision} = \text{recall} = \text{tool-F1} = 1.0$: flawless. But the *order* is dangerous, and edit distance flags it immediately — you must substitute both positions (or delete-then-insert), giving distance 2, not 0. Use set-based metrics when only the *choice* of tools matters; use edit distance when the *order*

of tools matters. Never report one and call it the other.

6 Step 5 — two axes: did it reach the goal, did it take a sensible path

Module 3.0’s central dichotomy is that *behavioral* eval and *trajectory* eval measure different things, and they routinely disagree. Edit distance is a trajectory metric; it says nothing about whether the goal was reached. Lay the two axes side by side on the module’s own Agent A vs. Agent B example (gold path [crm_lookup, refund, final_answer]):

Agent	steps	end-state	edit dist. to gold	traj. sim	behavioral
Agent A	3	refunded	0	1.000	PASS
Agent B	9	refunded	6	0.333	PASS

Both agents *succeed* — the refund is issued either way, so behavioral eval passes both and would happily ship either. But their trajectories could not be more different: Agent A walks the gold path exactly (distance 0, similarity 1.0), while Agent B wanders through web searches, a duplicate `crm_lookup`, and two `email_user` calls, landing 6 edits from gold (similarity 0.333). Same goal reached; wildly different path taken. ✓

Intuition

“Did it reach the goal?” and “did it take a sensible path?” are *orthogonal* axes. An agent can score a perfect behavioral PASS on a path that is a production cost bomb — right answer, wasteful route. That is precisely why Module 3.0 insists you need *both* lenses: behavioral eval alone would green-light Agent B and you would discover the bill only in production. Edit distance is the cheapest quantitative way to see the path quality that the end-state hides.

7 The DP table, drawn

	ϵ	search	summ.
ϵ	0	1	2
search	1	0	1
lookup	2	1	1
summ.	3	2	1

Annotations: A green arrow points down to the cell (search, search) containing 0. An orange arrow points down to the cell (lookup, search) containing 1, with the text "del lookup" next to it. A green arrow points from the cell (summ., search) containing 2 to the cell (summ., summ.) containing 1, with the text "match summ." next to it.

$$\text{distance} = 1 \text{ (one deletion); } \text{sim} = 1 - \frac{1}{3} = 0.667$$

The shaded diagonal (green matches, one orange delete) is the cheapest route from top-left to bottom-right: match **search**, delete **lookup**, match **summarize**. Its total cost is the boxed bottom-right cell. The picture is Equation (1), swept once across the grid.

8 Trajectory-distance cheat-sheet

Edit operations	insert (+1), delete (+1), substitute (+1), match (+0)
DP recurrence	$d[i][j] = \min(d[i-1][j]+1, d[i][j-1]+1, d[i-1][j-1]+1[a_i \neq b_j])$
Base cases	$d[i][0] = i, \quad d[0][j] = j$
Edit distance	$d[a][b]$ (the bottom-right cell)
Normalized similarity	$\text{sim} = 1 - \frac{d[a][b]}{\max(a , b)}$
Range	$0 \leq \text{distance} \leq \max(a , b); \quad 0 \leq \text{sim} \leq 1$
Order-agnostic set metric	$\text{precision} = \frac{ U \cap G }{ U }, \text{recall} = \frac{ U \cap G }{ G }, \text{F1} = \frac{2PR}{P+R}$
Use edit distance when	the <i>order</i> of tool calls matters
Use set P/R/F1 when	only the <i>choice</i> of tools matters (order-free)

9 An honest word about these numbers

The two sequences here ([**search**, **lookup**, **summarize**] vs. [**search**, **summarize**]) are illustrative, picked tiny so the whole DP table fits on one line and you can check every cell by eye — exactly as the cost-by-hand document used round token sizes no one would ship. Real trajectories run to dozens of steps, and you must decide what counts as “equal”: just the tool name (what we did here), or the tool name *plus* its arguments, which makes the comparison stricter. You may also weight the operations unequally (a wrong-tool substitution is often worse than a redundant insert). What is *not* illustrative is the structure: edit distance is the right shape whenever the output is an ordered sequence with multiple valid orderings, and the choice between it and set-based F1 is exactly the choice between “order matters” and “only the set matters.” Change the sequences and the numbers move; the recurrence in Equation (1) and the order-vs-set distinction do not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. It fills the DP table programmatically, prints it, reads off the distance and similarity, and computes the order-agnostic precision/recall/F1 for the same example — plus the order counterexample and the Agent A/B figures.

```

1 # trajectory_edit_distance.py -- reproduces every number in the document.
2
3 def levenshtein(a, b):
4     n, m = len(a), len(b)
5     dp = [[0] * (m + 1) for _ in range(n + 1)]
6     for i in range(n + 1):
7         dp[i][0] = i # delete all of a[:i]
8     for j in range(m + 1):
9         dp[0][j] = j # insert all of b[:j]

```

```

10     for i in range(1, n + 1):
11         for j in range(1, m + 1):
12             sub = 0 if a[i - 1] == b[j - 1] else 1
13             dp[i][j] = min(
14                 dp[i - 1][j] + 1, # delete a[i-1]
15                 dp[i][j - 1] + 1, # insert b[j-1]
16                 dp[i - 1][j - 1] + sub, # substitute / match
17             )
18     return dp
19
20 def distance(a, b):
21     dp = levenshtein(a, b)
22     return dp[len(a)][len(b)]
23
24 def similarity(a, b):
25     return 1 - distance(a, b) / max(len(a), len(b))
26
27 def set_prf1(used, gold):
28     U, G = set(used), set(gold)
29     inter = len(U & G)
30     p = inter / len(U)
31     r = inter / len(G)
32     f1 = 2 * p * r / (p + r)
33     return p, r, f1
34
35 # --- main example: fill and print the full DP table ---
36 actual = ["search", "lookup", "summarize"]
37 gold = ["search", "summarize"]
38 dp = levenshtein(actual, gold)
39 header = ["eps"] + [t[:4] for t in gold]
40 print("DP table (rows = actual, cols = gold):")
41 print(" " + " ".join(f"{h:>4}" for h in header))
42 for i in range(len(dp)):
43     label = "eps" if i == 0 else actual[i - 1][:4]
44     row = " ".join(f"{dp[i][j]:>4}" for j in range(len(dp[0])))
45     print(f"{label:>5} {row}")
46
47 d = distance(actual, gold)
48 print(f"edit_distance = {d}")
49 print(f"similarity = 1 - {d}/{max(len(actual), len(gold))} = "
50       f"{similarity(actual, gold):.4f}")
51
52 # --- order-agnostic set metrics for the same example ---
53 p, r, f1 = set_prf1(actual, gold)
54 print(f"precision={p:.4f} recall={r:.4f} tool_F1={f1:.4f}")
55
56 # --- order counterexample: same set, wrong order ---
57 p1 = ["crm_lookup", "refund"]
58 p2 = ["refund", "crm_lookup"]
59 pp, rr, ff = set_prf1(p1, p2)
60 print(f"order: set P=R=F1={pp:.1f}/{rr:.1f}/{ff:.1f} but "
61       f"edit_distance={distance(p1, p2)}")
62
63 # --- Agent A vs Agent B against the gold path (Module 3.0) ---
64 gold_path = ["crm_lookup", "refund", "final_answer"]
65 agent_a = ["crm_lookup", "refund", "final_answer"]
66 agent_b = ["web_search", "calculator", "crm_lookup", "email_user",
67            "crm_lookup", "shipping_status", "refund", "email_user",
68            "final_answer"]
69 print(f"Agent A: dist={distance(agent_a, gold_path)} "
70       f"sim={similarity(agent_a, gold_path):.3f}")
71 print(f"Agent B: dist={distance(agent_b, gold_path)} "
72       f"sim={similarity(agent_b, gold_path):.3f}")
73

```

```
74 # Expected output:
75 # DP table (rows = actual, cols = gold):
76 # eps sear summ
77 # eps 0 1 2
78 # sear 1 0 1
79 # look 2 1 1
80 # summ 3 2 1
81 # edit_distance = 1
82 # similarity = 1 - 1/3 = 0.6667
83 # precision=0.6667 recall=1.0000 tool_F1=0.8000
84 # order: set P=R=F1=1.0/1.0/1.0 but edit_distance=2
85 # Agent A: dist=0 sim=1.000
86 # Agent B: dist=6 sim=0.333
```

— end of document —